Here is the **Standard Operating Procedure (SOP)** for executing the Minimum Viable Product (MVP) of your Spy Satellite Simulator using your AI IDE (Antigravity / Cursor).
This SOP is designed to prevent AI hallucinations, maintain strict control over the codebase, and ensure you get a working project without overwhelming the AI.

📋 STANDARD OPERATING PROCEDURE (SOP)

Objective: Build and successfully render a live, moving satellite on a 3D Earth in the browser.
Tool: Antigravity IDE (or similar Agentic AI coding assistant).

🛠 PHASE 1: Workspace Preparation (The "Context Injection")

Before writing a single line of code, you must prime the AI's brain.

1. **Create the master file:** Open your IDE and create a new file in the root directory called `MVP-Plan.md`.
2. **Inject the context:** Paste the exact **MVP Scope** (from my previous response) into this file and save it.
3. **Set the boundaries:** Open the AI Chat/Agent panel and send this exact system prompt:
   "Read `MVP-Plan.md`. This is our absolute source of truth. We will build this strictly one step at a time. Acknowledge that you have read it and are ready for Step 1. Do not write code yet."

🔄 PHASE 2: The Execution Loop (Strict Step-by-Step)

You will use this loop for every single step of the MVP. **Rule #1: Never let the AI do two steps at once.**
Execution: Step 1 (The Bare Canvas)

- **Your Prompt:** *"Execute Step 1 from `MVP-Plan.md`. Run the terminal commands to initialize the Next.js App Router project, install `cesium` and `resium`. Then, create the basic full-screen `<Viewer>` component. Keep the UI entirely clean (disable base layer picker, timeline, animation widgets)."*
- **Verification:** Run `npm run dev`. Open `localhost:3000`. You must see a blank Cesium globe. *Do not proceed if the globe doesn't load.*

Execution: Step 2 (The Photorealistic Globe)

- **Your Prompt:** *"The base viewer is working. Now execute Step 2. Integrate Google Maps Photorealistic 3D Tiles using Cesium Ion. Make sure dynamic lighting is enabled based on the current UTC time."*
- **Verification:** Check the browser. You should now see high-fidelity Google Earth 3D tiles with realistic sun/shadow lighting.

Execution: Step 3 (The Math Engine)

- **Your Prompt:** *"The 3D tiles are working perfectly. Execute Step 3. Install `satellite.js`. Create a new utility file called `orbitalMath.ts`. Write a function that takes the ISS TLE string and the current time, and returns the Geodetic coordinates (Latitude, Longitude, Altitude). IMPORTANT: Ensure you convert radians to degrees before returning the values."*

- **Verification:** Ask the AI to write a quick `console.log` in the component to print the coordinates. Check your browser console. You should see numbers like `Lat: 45.2, Lon: -120.4, Alt: 400000`.

Execution: Step 4 (Rendering & Movement)

- **Your Prompt:** *"The math engine is outputting correct coordinates. Execute Step 4. Use Resium to create an `<Entity>` with a `<PointGraphics>` or 3D model to represent the satellite. Tie the position to a React state or Cesium Clock tick so that it updates every 1 second using the `orbitalMath.ts` function. The satellite should physically move across the globe."*
- **Verification:** Watch the screen. You should see a dot (or model) moving smoothly across the 3D globe in real-time.

🚨 PHASE 3: The Debugging Protocol

When (not if) the AI makes a mistake, follow this protocol to fix it without breaking the rest of the code:

1. **If the AI hallucinates Resium syntax (React errors):**
   - **Do not let it guess.** * **Prompt:** *"You used the wrong syntax for Resium. Please search the official documentation at `https://resium.reearth.io/` for the correct way to implement [Component Name] and fix the code."*
2. **If the satellite is in the wrong place (Math errors):**
   - AI models notoriously confuse radians and degrees.
   - **Prompt:** *"The satellite is rendering inside the earth or at the wrong coordinates. Check `orbitalMath.ts`. Ensure that the output from `satellite.js` is correctly converted from ECI to Geodetic, and verify that radians are being explicitly converted to degrees before passing them to Cesium."*
3. **If the browser crashes / runs out of memory:**
   - **Prompt:** *"The Cesium viewer is causing a memory leak. Ensure we are not creating a new `<Viewer>` instance on every React re-render. Check the `useEffect` hooks and ensure proper cleanup."*

🏁 FINAL SIGN-OFF

Once Step 4 is verified and you have a moving satellite on a 3D Google Maps globe, **STOP**. Commit your code to GitHub. You now have a working, bug-free MVP.
Only after this is secure should you ask the AI to start adding UI panels, dark mode, or external API data.